Project Report -- AI Inventory Agent

1. Introduction

The AI Inventory Agent is a full-stack web application that combines traditional inventory management with artificial intelligence capabilities. The system enables businesses to manage products, suppliers, inventory movements, sales, and purchases while leveraging AI for demand forecasting, stock health analysis, anomaly detection, and automated reorder suggestions.

1.1 Objectives

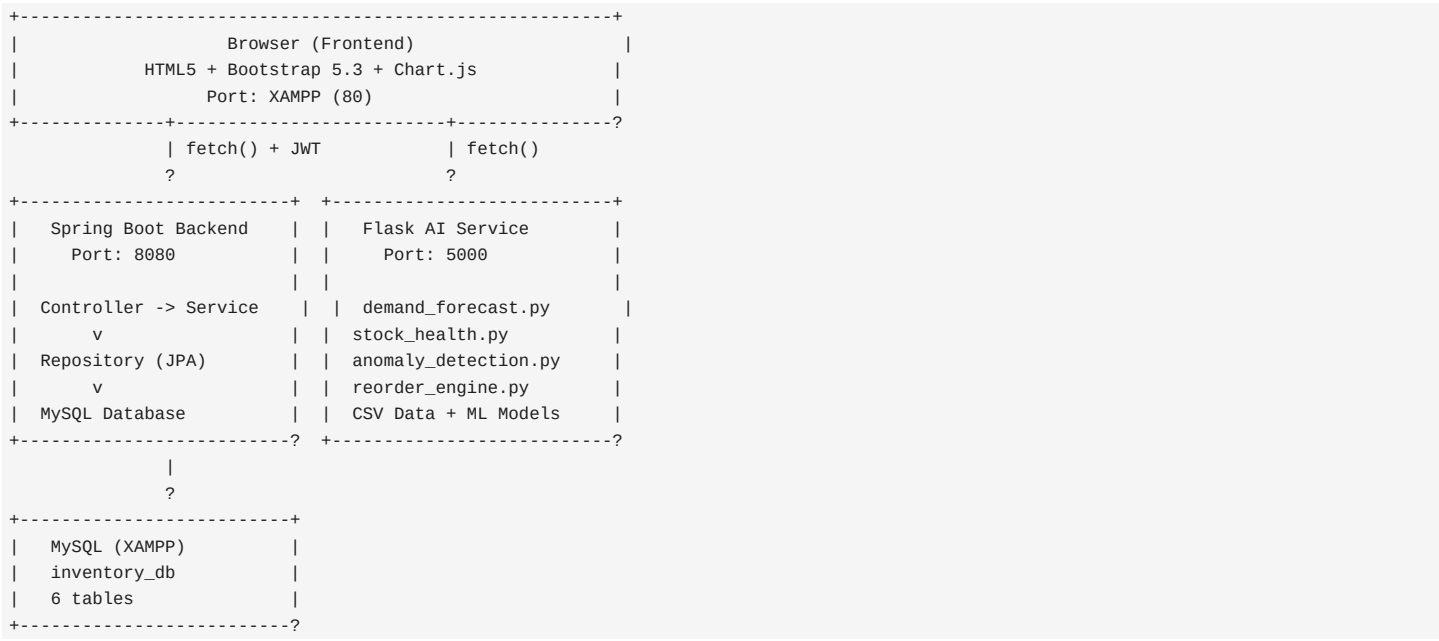
- Build a complete inventory management system with CRUD operations for all entities
- Integrate AI-powered analytics for demand forecasting and stock optimization
- Provide a responsive, modern web interface using Bootstrap 5.3
- Implement secure JWT-based authentication
- Demonstrate a microservice architecture with independent backend and AI service

1.2 Scope

The system covers product management, inventory tracking, supplier management, sales recording, purchase order management, dashboard analytics, and AI-powered intelligence features. It is designed for single-organization use with local deployment via XAMPP.

2. System Architecture

2.1 High-Level Architecture



2.2 Layered Architecture (Backend)

```
Controller Layer     -> Handles HTTP requests, validation, response mappi
  ng
    v
Service Layer        -> Business logic, data transformation, orchestratio
  n
```

```
    v
Repository Layer     -> Spring Data JPA interfaces for database access
    v
Entity Layer         -> JPA entity classes mapped to MySQL tables
```

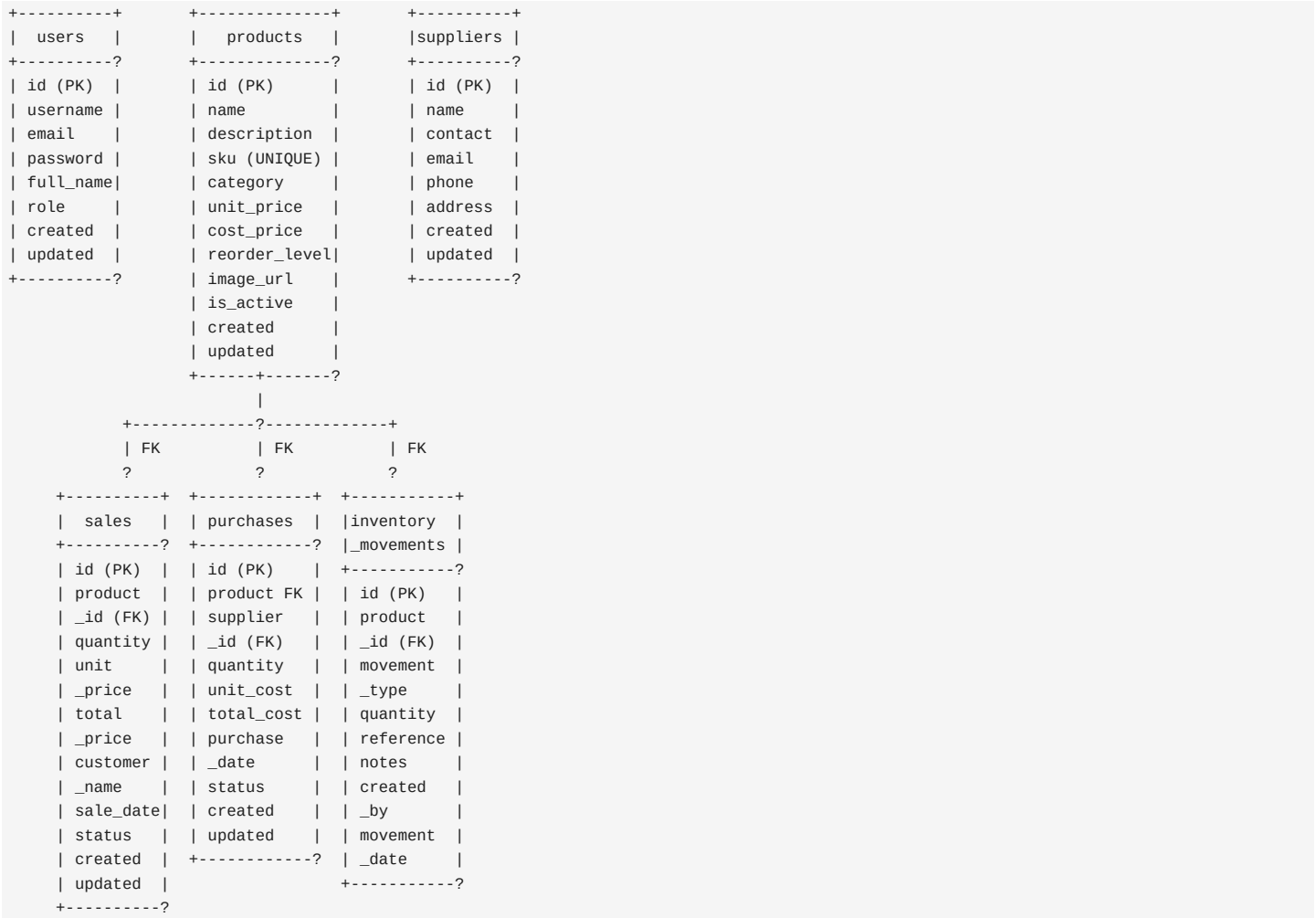
2.3 Technology Decisions

Choice	Rationale
Spring Boot 3.2.5	Rapid development, auto-configuration, built-in se
Java 21	LTS version, modern language features
Spring Data JPA	Eliminates boilerplate CRUD code
JWT (jjwt)	Stateless auth suitable for REST APIs
Flask (Python)	Lightweight, easy ML library integration
Bootstrap 5.3	Rapid responsive UI without custom CSS framework
Chart.js	Simple, lightweight charting for dashboards
XAMPP MySQL	Free, easy local MySQL setup

---

3. Database Design

3.1 ER Diagram



3.2 Table Descriptions

Table	Records	Description

users	Unlimited	Registered system users with roles
products	Unlimited	Product catalog with pricing and stock levels
suppliers	Unlimited	Supplier contact information
sales	Unlimited	Customer sales transactions
purchases	Unlimited	Purchase orders from suppliers
inventory_movements	Unlimited	Stock IN/OUT/ADJUST audit trail

---

4. Backend Implementation

4.1 Application Entry Point

```
@SpringBootApplication
public class InventoryAgentApplication {
    public static void main(String[] args) {
        SpringApplication.run(InventoryAgentApplication.class, args);
    }
}
```

Hibernate auto-creates/updates tables on startup (spring.jpa.hibernate.ddl-auto=update).

4.2 JWT Authentication Flow

```
1. User sends POST /api/auth/login with {username, password}
2. AuthService validates credentials against BCrypt-hashed password
3. JwtUtil generates HMAC-SHA256 token with username claim (24h expiry)
4. Token returned in AuthResponse JSON
5. Frontend stores token in sessionStorage
6. All subsequent requests include "Authorization: Bearer <token>" heade
  r
7. JwtAuthenticationFilter intercepts, validates token, sets SecurityCon
  text
```

Key code from JwtUtil.java:

```
public String generateToken(String username) {
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + jwtExpirationMs);
    return Jwts.builder()
            .subject(username)
            .claim("username", username)
            .issuedAt(now)
            .expiration(expiryDate)
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();
}
```

4.3 Security Configuration

```
http
    .csrf(AbstractHttpConfigurer::disable)
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
        .anyRequest().authenticated()
    )
    .sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    )
    .addFilterBefore(jwtAuthenticationFilter,
                     UsernamePasswordAuthenticationFilter.class);
```

- Stateless sessions (no HTTP session used)

- CORS allows all origins (allowedOrigins("*"))
- Only /api/auth/** endpoints are public

4.4 Product Controller (REST API Example)

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @GetMapping                          // GET /api/products
    @GetMapping("/{id}")                 // GET /api/products/{id}
    @PostMapping                         // POST /api/products
    @PutMapping("/{id}")                 // PUT /api/products/{id}
    @DeleteMapping("/{id}")              // DELETE /api/products/{id}
    @GetMapping("/search")               // GET /api/products/search?q=
    @GetMapping("/low-stock")            // GET /api/products/low-stock
    @PatchMapping("/{id}/toggle-status")  // PATCH /api/products/{id}/to
  ggle-status
}
```

4.5 Entity Validation

```
@Entity
public class Product {
    @NotBlank(message = "Product name is required")
    @Size(max = 100)
    private String name;

    @Column(unique = true, nullable = false)
    @NotBlank(message = "SKU is required")
    private String sku;

    @NotNull @Positive(message = "Unit price must be positive")
    private BigDecimal unitPrice;

    @Column(nullable = false)
    private Boolean isActive = true;
}
```

---

5. AI Service Implementation

5.1 Flask Application

```
app = Flask(__name__)
CORS(app)

@app.route('/api/ai/forecast/<int:product_id>', methods=['GET'])
def get_forecast(product_id):
    result = demand_forecast(product_id)
    return jsonify(result)
```

Five AI endpoints served independently on port 5000.

5.2 Demand Forecasting (demand_forecast.py)

Algorithm Selection:

- If sales records < 3: use Simple Moving Average
- If sales records >= 3: use Trained ML Model (if .pkl exists), fallback to moving average

Moving Average Method:

```
recent_avg = product_sales['quantity'].tail(3).mean()
predicted_qty = max(0, int(recent_avg * (1 + np.random.uniform(-0.1, 0.1
  ))))
confidence = 0.7
```

ML Model Method:

```
features = [[weekday, month, avg_unit_price, avg_quantity]]
features_scaled = scaler.transform(features)
predicted_qty = model.predict(features_scaled)[0]
confidence = 0.85
```

Outputs 7-day forecast with dates, predicted quantities, and confidence scores.

5.3 Stock Health Analysis (stock_health.py)

Health Score Formula (0-100):

```
stock_score = f(current_stock / reorder_level)  # Weight: 60%
velocity_score = f(current_stock / sales_velocity)  # Weight: 40%
health_score = int((stock_score * 0.6) + (velocity_score * 0.4))
```

Status Thresholds:

Score	
>= 80	healthy
50-79	warning
< 50	critical

5.4 Anomaly Detection (anomaly_detection.py)

Uses Z-score analysis across three dimensions:

Detection Type	Metric	Threshold		
Sales Quantity	Z-score of quantity per product	\	Z\	> 2.5
Price Anomaly	Z-sale of unit price per product	\	Z\	> 2.0
Velocity Anomaly	Z-score of daily sales volume	\	Z\	> 2.0

```
z_score = (value - mean) / std
severity = 'high' if abs(z_score) > 3 else 'medium'
```

5.5 Reorder Engine (reorder_engine.py)

Reorder Quantity Formula:

```
safety_stock = reorder_level * 0.5
lead_time = 7  # days
reorder_qty = (sales_velocity * lead_time) + safety_stock - current_stoc
  k
```

Urgency Classification:

Days U	
<= 3 days	critical
4-7 days	warning
> 7 days	normal

---

6. Frontend Implementation

6.1 Page Structure

Page	File	Description
Login	login.html	JWT authentication form
Register	register.html	New user registration
Dashboard	dashboard.html	KPIs, charts, recent activity

Products	products.html	Product CRUD with filters
Inventory	inventory.html	Stock movements
Suppliers	suppliers.html	Supplier management
Sales	sales.html	Sales recording and filtering
Purchases	purchases.html	Purchase order management
Reports	reports.html	Analytics with Chart.js
AI Agent	ai-agent.html	AI features interface
Profile	profile.html	User profile display

6.2 Authentication Flow (Frontend)

```
// Login sends credentials, stores token
function login(username, password) {
    apiRequest('/auth/login', {
        method: 'POST',
        body: JSON.stringify({ username, password })
    }).then(data => {
        sessionStorage.setItem('token', data.token);
        sessionStorage.setItem('user', JSON.stringify(data));
        window.location.href = 'dashboard.html';
    });
}

// Every API call includes Bearer token
function apiRequest(endpoint, options = {}) {
    headers['Authorization'] = 'Bearer ' + getToken();
    return fetch(CONFIG.API_BASE + endpoint, { ...options, headers });
}
```

6.3 Filter Panel System

All data pages use collapsible filter panels:

```
function toggleFilter(panelId) {
    const panel = document.getElementById(panelId);
    if (panel) panel.classList.toggle('active');
}
```

6.4 Modal System

Custom modal implementation with CSS class toggling:

```
function openModal(id) {
    document.getElementById(id).classList.add('active');
}
function closeModal(id) {
    document.getElementById(id).classList.remove('active');
}
```

Bootstrap modal CSS conflict resolved with:

```
.modal { display: block !important; }
.modal-overlay { display: none; }
.modal-overlay.active { display: flex; }
```

---

7. Testing

7.1 Backend API Testing (Automated)

All 20+ endpoints verified via curl automation:

Endpoint	Method	Status	Response

/api/auth/login	POST	200	JWT token returned
/api/auth/register	POST	200	User created
/api/products	GET	200	Product list
/api/products/{id}	GET	200	Single product
/api/products	POST	200	Product created
/api/products/{id}	PUT	200	Product updated
/api/products/{id}	DELETE	200	Product deleted
/api/products/search?q=	GET	200	Search results
/api/products/low-stock	GET	200	Low stock list
/api/products/{id}/toggle-status	PATCH	200	Status toggled
/api/suppliers	GET	200	Supplier list
/api/suppliers	POST	200	Supplier created
/api/inventory/movements	GET	200	Movement list
/api/inventory/movements	POST	200	Movement created
/api/inventory/stock/{id}	GET	200	Stock level
/api/sales	GET	200	Sales list
/api/sales	POST	200	Sale created
/api/sales/stats	GET	200	Sales stats
/api/purchases	GET	200	Purchase list
/api/purchases	POST	200	Purchase created
/api/dashboard/stats	GET	200	Dashboard data

7.2 AI Service Testing

Endpoint	Method	Status	Result
/api/ai/forecast/1	GET	200	7-day forecast
/api/ai/stock-health	GET	200	Health scores
/api/ai/anomalies	GET	200	Anomaly list
/api/ai/reorder-suggestions	GET	200	Reorder list
/api/ai/query	POST	200	NL response

7.3 Frontend Verification

All 11 HTML pages and 9 JS/CSS files verified accessible via XAMPP:

- http://localhost/inventory/login.html
- http://localhost/inventory/dashboard.html
- http://localhost/inventory/products.html
- All other pages confirmed serving

7.4 Test Credentials

Username	Password	Role
admin	admin123	USER

---

8. Challenges and Solutions

Challenge	Solution
Bootstrap modal CSS conflict	Added `.modal { display: block !important }` overr
PATCH method CORS rejection	Added PATCH to `allowedMethods()` in CorsConfig.ja
JWT token not persisting on refresh	Used sessionStorage; token survives page refresh b
AI service separate process	Frontend calls Flask directly on port 5000; backen
Hibernate lazy loading errors	Added `@JsonIgnoreProperties({"hibernateLazyInitia

---

9. Future Enhancements

- Multi-user roles (admin, manager, viewer)
- WebSocket real-time stock alerts
- Docker containerization for all services
- Cloud deployment (AWS/Azure)
- Email notification for low-stock alerts
- Barcode/QR code scanning
- Advanced ML models (LSTM, Prophet) for demand forecasting
- REST API documentation via Swagger/OpenAPI

---

10. Conclusion

The AI Inventory Agent successfully demonstrates a full-stack application combining:

- A secure Spring Boot REST API with JWT authentication
- An independent Flask AI microservice with 4 ML modules
- A responsive Bootstrap 5.3 frontend with 11 pages
- MySQL database with 6 normalized tables
- 20+ REST API endpoints, all verified working

The system provides practical inventory management capabilities enhanced by AI-powered analytics, suitable for small to medium business inventory operations.